

WAFT Documentation Team

WAFT Demo Walkthrough

Complete Guide to WAFT PDF Tools

Version 1.0

January 11, 2026

Compiled by WAFT System

2 Sections | 3 Documents

Table of Contents

Introduction

Overview of WAFT PDF tools

WAFT Demo Walkthrough - Introduction WAFT Documentation Team | January 11, 2026

WAFT Demo Walkthrough - Introduction (Printer Friendly)

WAFT Documentation Team | January 11, 2026

Demo Examples

Example demonstrations of WAFT capabilities

Waft Demo Booklet WAFT System | January 11, 2026

INTRODUCTION

Overview of WAFT PDF tools

WALKTHROUGH WT-001

WAFT DEMO WALKTHROUGH

Introduction to WAFT PDF Tools

DEMONSTRATION

Issued by: WAFT Documentation Team

Date: January 11, 2026

WELCOME TO THE WAFT PDF TOOLS WALKTHROUGH

This walkthrough demonstrates WAFT's powerful PDF generation and organization tools. You'll see how WAFT uses its own templates to document itself, creating a recursive self-improvement loop.

WHAT YOU'LL LEARN

- How to use the Field Guide template
- How to create printer-friendly versions
- How to combine PDFs using the Binder system
- How to showcase multiple templates
- How WAFT documents itself recursively

TOOL 1: FIELD GUIDE TEMPLATE

The Field Guide template creates operational manuals with a military field manual aesthetic. It features:

DEMONSTRATION

FIELD GUIDE FEATURES

- ☐ Two-column layout support
- ☐ Warning and caution boxes
- ☐ Equipment checklists
- ☐ Step-by-step procedures
- ☐ Professional tables
- ☐ Rugged, practical design

Usage Example

```
from src.waft.templates.field_guide import generate_field_guide
from pathlib import Path

generate_field_guide(
    title="My Field Guide",
    content="<h2>Content</h2><p>Your HTML here</p>",
    output_path=Path("output.pdf"),
    series="FIELD GUIDE",
    number="FG-001"
)
```

TOOL 2: PRINTER-FRIENDLY CONVERSION

WAFT includes a helper script to convert any template to printer-friendly (black and white) versions. This is essential for:

- Cost-effective printing
- Professional documentation
- Accessibility (works with all printers)
- Reduced file sizes

IMPORTANT

The printer-friendly version removes all colors and replaces them with black borders, white backgrounds, and text labels. All content is preserved, just optimized for black-and-white printing.

TOOL 3: BINDER SYSTEM

The Binder system combines multiple PDF documents into a single, organized booklet with:

- 1 Cover page with title and metadata
- 2 Table of contents listing all sections
- 3 Section dividers between document groups
- 4 All documents merged in order

Usage Example

```
from src.waft.binder import Binder, DocumentEntry
from pathlib import Path

binder = Binder(
    title="My Binder",
    subtitle="Document Collection"
)

section = binder.add_section("Section 1")
section.add_document(DocumentEntry(
    path=Path("doc1.pdf"),
    title="Document 1"
))

binder.generate(Path("binder.pdf"))
```

TOOL 4: TEMPLATE SHOWCASE

WAFT includes 12 professional document templates:

Table 1: WAFT Document Templates

Template	Use Case	Status
Field Guide	Operational manuals	✓ Production
Lab Notes	Research logs	✓ Production
TM Report	Technical memos	✓ Production
Personal Memo	Staff communications	✓ Production
Simple Scientific	Academic papers	✓ Production

Eldritch Journal	Horror/madness themes	✓ Production
Screenplay	Film/TV scripts	✓ Production
Heartfelt Letter	Personal letters	✓ Production
Invoice/Contract	Business documents	✓ Production
Code Documentation	Technical docs	✓ Template
Children's Storybook	Kids' books	✓ Template
Newspaper	News front pages	✓ Template

THE RECURSIVE LOOP

WAPT Documenting WAPT

This walkthrough itself demonstrates WAPT's recursive capabilities:

1. **WAPT generates this walkthrough** using its Field Guide template
2. **WAPT creates a printer-friendly version** using the helper script
3. **WAPT combines everything** using the Binder system
4. **WAPT documents the process** in this very document

This creates a feedback loop where WAPT improves by understanding itself better.

NEXT STEPS

- 1 Review the generated walkthrough booklet
- 2 Check out the printer-friendly version
- 3 Explore the individual template examples
- 4 Try creating your own documents using WAPT templates

REMEMBER

WAPT's tools are designed to work together. Use templates to generate documents, use the binder to organize them, and use the printer-friendly helper to optimize for printing. All while WAPT tracks its work in the `_pyrite` system.

WALKTHROUGH WT-001

WAFT DEMO WALKTHROUGH

Introduction to WAFT PDF Tools (Printer Friendly)

DEMONSTRATION

Issued by: WAFT Documentation Team

Date: January 11, 2026

WELCOME TO THE WAFT PDF TOOLS WALKTHROUGH

This walkthrough demonstrates WAFT's powerful PDF generation and organization tools. You'll see how WAFT uses its own templates to document itself, creating a recursive self-improvement loop.

[NOTE] WHAT YOU'LL LEARN

- How to use the Field Guide template
- How to create printer-friendly versions
- How to combine PDFs using the Binder system
- How to showcase multiple templates
- How WAFT documents itself recursively

TOOL 1: FIELD GUIDE TEMPLATE

The Field Guide template creates operational manuals with a military field manual aesthetic. It features two-column layout, warning boxes, checklists, procedures, and professional tables.

DEMONSTRATION

TOOL 2: PRINTER-FRIENDLY CONVERSION

WAFT includes a helper script to convert any template to printer-friendly (black and white) versions. This is essential for cost-effective printing, professional documentation, accessibility, and reduced file sizes.

[CAUTION] IMPORTANT

The printer-friendly version removes all colors and replaces them with black borders, white backgrounds, and text labels. All content is preserved, just optimized for black-and-white printing.

TOOL 3: BINDER SYSTEM

The Binder system combines multiple PDF documents into a single, organized booklet with cover page, table of contents, section dividers, and all documents merged in order.

TOOL 4: TEMPLATE SHOWCASE

WAFT includes 12 professional document templates for various use cases including field guides, lab notes, technical memos, personal memos, academic papers, and more.

THE RECURSIVE LOOP

[NOTE] WAFT DOCUMENTING WAFT

This walkthrough itself demonstrates WAFT's recursive capabilities:

1. WAFT generates this walkthrough using its Field Guide template
2. WAFT creates a printer-friendly version using the helper script
3. WAFT combines everything using the Binder system
4. WAFT documents the process in this very document

This creates a feedback loop where WAFT improves by understanding itself better.

NEXT STEPS

- 1 Review the generated walkthrough booklet
- 2 Check out the printer-friendly version
- 3 Explore the individual template examples

4

Try creating your own documents using WAFT templates

DEMO EXAMPLES

Example demonstrations of WAFT capabilities

WAFT

Meta-Cognitive Demonstration

World Architecture Framework & Templates

2026-01-11 07:56:00

Introduction

This booklet documents a demonstration of WAFT's meta-cognitive capabilities. WAFT is a self-documenting, self-modifying meta-framework that tracks its own work and enables continuity across AI sessions through epistemic memory.

What You Witnessed

- **Basic file organization** - Simple folder cleanup (2022 ChatGPT level)
- **Work effort management system** - The `_pyrite` system for tracking intellectual labor
- **Epistemic memory** - How WAFT remembers what it knows
- **Meta-cognitive perspective-taking** - How AI systems can "wear" previous perspectives
- **Recursive self-improvement foundation** - The basis for continuous enhancement

Demo Structure

The demonstration created the following folder structure:

```
└─ demo_output/
  │   └─ .waft_template/
  │       └─ README.md/
  │   └─ README.md/
  │   └─ data/
  │       │   └─ config.yaml/
  │       │   └─ data.json/
  │       │   └─ results.csv/
  │   └─ documents/
  │       │   └─ document1.txt/
  │       │   └─ log.txt/
  │       │   └─ notes.md/
  │       │   └─ readme.txt/
  │   └─ scripts/
  │       │   └─ script.py/
  │       │   └─ test.py/
  │   └─ temp/
  │       │   └─ old_backup.bak/
  │       │   └─ temp_file.tmp/
  └─ tools/
      │   └─ _pyrite/
      │       │   └─ active/
      │       │       │   └─ demo_journal.md/
      │       │       │   └─ demo_work_effort.md/
      │       │   └─ backlog/
```

```
| └─ standards/  
└─ meta_cognition_explanation.md/
```

Meta-Cognition: The Core Concept

Why _pyrite?

So that WAFT can track on its own what it knows and what it doesn't.

The work efforts ticketing system acts as a sort of **rudimentary epistemic memory** - a journal that any LLM can pick up and wear like a pair of glasses to see how the previous AI saw its world.

This is perspective taking.

This is a very, very, very basic, very very very simple form of LLM meta-cognition across architectures using a work efforts and journaling system to track "thoughts" or **intellectual labor quanta** in the form of text in the WAFT system, which can self-modify and recursively self-improve based on external and internal feedback.

How It Works

1. **Work Efforts:** Track discrete units of intellectual labor
2. **Journaling:** Record thoughts, learnings, and observations
3. **Perspective Taking:** New AI instances can "put on" the previous AI's perspective by reading the work efforts and journals

Why It Matters

This enables:

- **Continuity:** Knowledge persists across AI sessions

- **Self-Awareness:** System knows what it knows
- **Recursive Improvement:** System can improve based on its own observations
- **Meta-Cognition:** Thinking about thinking

The Recursive Loop

1. AI does work → Creates work effort
2. AI reflects → Writes journal entry
3. AI documents → Records what it learned
4. Next AI reads → Understands previous context
5. Next AI continues → Builds on previous knowledge
6. Cycle repeats → Continuous improvement

Intellectual Labor Quanta

Each work effort, journal entry, or documentation piece represents a **quantum of intellectual labor** - a discrete unit of thought and work that can be tracked, measured, and built upon.

Cross-Architecture Meta-Cognition

This system works across different AI architectures because it's based on **text** - the universal interface. Any LLM can read and understand:

- Work effort descriptions
- Journal entries
- Documentation
- Status updates

This creates a form of **perspective-taking** where one AI can understand how another AI (or a previous version of itself) saw the world.

Next Steps

The demo folder structure created during this demonstration can serve as a **jumping off point for full installation of the WAFT system**. The organized structure, tools folder, and _pyrite system provide the foundation for:

- Organic system growth and expansion
- Work effort tracking and management
- Epistemic memory development
- Recursive self-improvement

Generated by WAFT - World Architecture Framework & Templates

This is WAFT tracking its own work, understanding its own state,
and enabling future AI instances to continue where this one left off.